

SchemaForge

Rehearse PostgreSQL migrations, prove what happened, stop before production

TrueFoundry × Polaris “Agents That Act” Hackathon · Project Summary · September 2026

The model proposes; tools measure; policy constrains; a human decides.

What it is

SchemaForge is a TrueForge agent that turns a natural-language database change into an evidence-backed migration decision. It inspects a live PostgreSQL 16 target, maps the dependency blast radius, runs the generated SQL inside a rollback-only shadow sandbox, checks explicit assertions and rollback equivalence, and then freezes at a human approval checkpoint. It is not a free-running SQL generator: nothing reaches production without a signed approval bound to the exact SQL and assertion set, followed by a second Allow/Deny gate in TrueForge.

Architecture

Two role-separated Streamable HTTP MCP servers, written in TypeScript, are attached to one TrueForge agent.

Component	Responsibility and credentials
schemaforge-core :3100	Schema inspection, bounded read-only queries, dependency analysis, shadow rehearsal, post-apply verification. Holds only the <code>sf_reader</code> (production, read-only) and <code>sf_shadow</code> credentials.
schemaforge-executor :3101	Exposes a single tool, <code>execute_migration</code> , marked <code>destructiveHint: true</code> . Holds the <code>sf_executor</code> write role (not a superuser) and the approval-verification secret; has no shadow access.
Approval CLI	<code>npm run approve</code> runs in the human’s terminal and mints a short-lived HMAC-SHA256 token. The signing key never enters the agent loop.
PostgreSQL :5433 / :5434	Production and shadow databases in Docker Compose, both seeded with the same deterministic data (500 users, 14 NULL emails, a duplicate-email group, FK and precision edge cases).

The credential split is structural, not a prompt promise: configuration validation fails fast if the core process is given the write URL or approval secret.

MCP tools

Tool	Annotation	Purpose
db_inspect_schema	read-only	Catalog snapshot and stable schema fingerprint
db_run_readonly_query	read-only	Bounded <code>SELECT</code> / read-only CTE via <code>sf_reader</code>
analyze_dependencies	read-only	Foreign keys, views, functions/triggers, indexes affected
rehearse_migration	write, non-destructive	Rollback-only shadow execution with observed evidence
verify_production	read-only	Expected fingerprint plus explicit postcondition assertions
execute_migration	destructive	Signed one-shot apply with exact SQL/assertions and pre-commit postconditions

End-to-end flow

1. The human states an objective; the agent inspects schema, data and dependencies (read-only).
2. The agent proposes forward SQL, rollback SQL and typed assertions; the SQL policy classifies them.
3. `rehearse_migration` runs everything on one shadow session and returns a verdict and fingerprints.
4. The agent publishes a frozen decision packet and stops.
5. A human mints a token bound to exact SQL, assertions, fingerprints and rehearsal ID (five-minute TTL).
6. TrueForge pauses the destructive call on an Allow/Deny card; the executor re-verifies, runs the approved assertions and applies atomically.

Safety properties

- **Fail-closed SQL policy.** Tier-3 operations, DML, procedural escape hatches, internal-ledger access, ambiguous escapes, unknown statements and mixed or non-transactional plans are rejected in code.
- **Deterministic sandbox.** Forward SQL, assertions, lock observation, fingerprints and rollback run on one shadow connection; an outer ROLLBACK always runs, even on success.
- **Assertions, not vibes.** Each check declares an expected outcome (`first_value_true`, `scalar_equals`, `returns_rows`, `returns_no_rows`); returning without error never passes.
- **Rollback equivalence.** Rollback SQL runs under a savepoint and must restore the exact baseline fingerprint.
- **Exact-action approval.** The HMAC covers SQL and assertion-set hashes, baseline and expected fingerprints, rehearsal ID, action, target, nonce, issue time, expiry and single-use intent.
- **Replay, drift and postcondition checks.** The nonce is claimed atomically in a production ledger; the apply aborts on baseline drift, then rolls back unless both the post-fingerprint and all approved assertions pass. Applies are serialized with an advisory lock under statement and lock timeouts.

How the verdict is decided

Computed by code in `rehearse_migration`; the model explains the verdict but cannot change it.

Evidence	Verdict
Policy rejects the SQL (prohibited, unsupported, non-transactional)	rejected before rehearsal
Forward SQL fails, or any assertion fails	DO_NOT_APPLY
Rollback supplied but does not restore the baseline fingerprint	DO_NOT_APPLY
No rollback supplied, or the plan is classified destructive	REVIEW
Rehearsal, assertions and rollback all verified	APPLY

Where production apply is refused

Code	Cause
TOKEN_MALFORMED / SIGNATURE_INVALID	Token unparseable or signed with the wrong key
HASH_MISMATCH / ASSERTIONS_MISMATCH / TARGET_MISMATCH	SQL or assertions edited after approval, or wrong target
TOKEN_EXPIRED / TOKEN_NOT_YET_VALID / TTL_EXCEEDED	Outside the validity window or TTL too long
REPLAY_DETECTED	Nonce already claimed in the ledger
SCHEMA_DRIFT	Production no longer matches the rehearsed baseline
POSTCONDITION_FAILED	Post-apply fingerprint differs; transaction rolled back
SQL_POLICY_REJECTED	Statement fails the policy re-check in the executor

Demo scenarios

A — knowing when to stop. “Make `users.email` NOT NULL and UNIQUE.” The agent observes 14 NULLs and a duplicate group; the naive migration fails rehearsal, the verdict is `DO_NOT_APPLY`, and no production tool is called.

B — safe action behind two gates. “Add an optional `phone VARCHAR(32)` column.” Rehearsal passes, rollback (`DROP COLUMN phone`) restores the baseline, and an `information_schema` assertion confirms the column. The human mints a token, TrueForge shows Allow/Deny, the executor applies atomically, and reusing the token returns `REPLAY_DETECTED`.

Stack and validation

TrueForge 0.2.1 harness, Node.js 22.14+, TypeScript MCP servers, PostgreSQL 16 in Docker Compose, Daytona sandbox for helper code. Validate with `npm run check`, `npm run build`, `npm test` and `docker compose config -quiet`. No mocked responses: databases, roles, rehearsal and migration are real.

Repository: <https://github.com/harshaxyZ/schemaforge> · MIT licence · Built with AI assistance (Kiro IDE); the human team owns product decisions.